Development Workflow — Executive Guide

Audience: Founder / leader without a day-to-day coding background.
Purpose: Plain-language map from idea → safe testing → live customers.

> **Big picture:** Your product has two "homes": the **website** (what users see) and the **database & backend** (where data and payments logic live). Moving a change to production means updating both, in the right order, after you have checked it in a safe copy of the system (your "dev" environment).

1. Planning with Claude (or another AI assistant)

This is the "what should we build and why" phase. You describe the business outcome; the assistant helps turn it into a clear plan.

1. **State the business problem** — e.g. "Payments show up too slowly on the customer dashboard" or "We need receipts to flow from the form into our system without manual spreadsheet work."
2. **Agree on success** — What will "done" look like for you and for customers? (e.g. "Balance updates within a minute of approval.")
3. **Review constraints** — Budget, timeline, legal/compliance, and what must *not* change (e.g. existing Odoo or payment gateway rules).
4. **Get a written plan** — A short document listing steps, main parts of the system touched, and risks. Treat this as your checklist, not as code.
5. **Assign ownership** — Who approves the plan, who implements (you with AI, a developer, or both), and who signs off before customers see it.

In plain terms: Planning with Claude is like agreeing on the blueprint and acceptance tests before anyone remodels the house.

2. Edits to the codebase

The "codebase" is the folder of files that defines your app and its connection to the database. Changes here are invisible to customers until those changes are *built*, *deployed*, and pointed at the live database.

1. **Work on a branch or copy** — Ideally, new work lives on a separate line of development (often called a "branch") so the live version stays untouched until you are ready.
2. **Implement the plan** — Screens, logic, database structure (migrations), and "serverless" functions (small programs that run in the cloud when something happens) are updated to match the plan.
3. **Keep changes small and reviewable** — Each chunk of work should map to one goal so you can test and roll back if needed.

4. **Save to Git** — Your hosting and tools (including Lovable) usually connect to **Git**, which is a history of every saved version. Pushing those saves to your central repository is how the team (and deployment tools) get the latest files.
5. **Do not assume "save = live"** — Saving code to the repository updates the *source of truth*. The public site and production database are updated only when you run your deployment steps (see sections 3 and 4).

In plain terms: Editing the codebase is updating the recipe; customers still get the old meal until you cook and serve the new one in the right kitchen (dev first, then production).

3. Testing and viewing in a development ("dev") environment

A **dev environment** is a safe copy: wrong buttons or bugs do not affect paying customers or real financial records (when you use a separate database or test data).

1. **Run the app locally or on a preview URL** — For this project, developers often use a command like starting the dev server on a laptop so the site opens in a browser with instant reload. Lovable may also offer a preview of the project before you "publish."
2. **Point the app at non-production data** — Use a **staging** or **development** Supabase project (or local database) so tests do not touch production customer data.
3. **Apply backend changes there first** — Database structure updates (migrations) and cloud functions should be deployed to the *same* environment you are testing against, not straight to production while you are still learning if the feature works.
4. **Walk through real scenarios** — Log in as a test customer, submit a form, approve a receipt, check balances and emails. Compare results to the plan in section 1.
5. **Fix issues and repeat** — Until the checklist passes, stay in dev. Only then proceed to production deployment.

In plain terms: Dev is the rehearsal stage. Nothing here should frighten customers or accountants.

4. Deploying to production

Production is what real users and real data use. Two parts usually deploy separately:

- **Frontend (the website)** — For Lovable-hosted projects, this is typically an explicit step such as **Share** → **Publish** inside Lovable. Pushing code to Git alone does not always replace that step for the live URL.
- **Backend (database + server logic)** — Supabase migrations and edge functions must be applied to your **production** Supabase project using your team's agreed process (dashboard, CLI, or CI/CD). Publishing the website in Lovable does not automatically run database migrations or deploy every function.

1. **Freeze scope** — Agree that dev testing is complete and what exactly is going live.
2. **Deploy backend first (or in the order your technical lead specifies)** — So the live site does not call APIs or tables that do not exist yet.
3. **Publish the frontend** — e.g. Lovable **Publish** so the public site serves the new build.

4. **Smoke-test production** — One careful pass on the live URL with a known test account (or read-only checks) to confirm critical paths.
5. **Monitor and communicate** — Watch for errors, support tickets, and payment anomalies; tell stakeholders when the release is complete.

In plain terms: Production deployment is opening the restaurant to guests after the staff-only rehearsal — two switches (website + backend), both intentional.

> **How to save this document as a PDF:** Open this file in **Chrome** or **Edge**, press **Ctrl+P** (Windows) or **Cmd+P** (Mac), choose **Save as PDF** as the printer, then save. On a phone, open the file from cloud storage and use the share/export-to-PDF option if available.

Document generated for the Lake City Customer Portal repository. Adjust names of tools (Lovable, Supabase, Git) only if your team uses different vendors; the sequence stays the same: plan → build in repo → test in dev → deploy backend and frontend to production.